

RUDP: Video Script for Three Speakers

Oct 8, 2026 · @Harsh

About this video

A 13-minute walkthrough for three speakers, built to match the 14-slide deck `RUDP_Presentation.pptx`. Slide numbers below refer to that deck.

Item	Detail
Course	Computer Networks
Project	RUDP: a reliable transport protocol built on UDP
Team	Harshavardhan Mallela (24BAI0274), Rohit Arya (24BCE0819), Shloak Sinha (24BDS0378)
Faculty	Anuradha G
Due	21 October 2026

Speaker	Parts	Slides
Harshavardhan	Opening, problem, what we built, demo	1–5
Rohit	Reliability, congestion control, literature	6, 7, 11
Shloak	Testing, results, problems, topology	8, 9, 10, 12
All three	Learnings, future work, close	13, 14

Spoken lines are written to be said aloud. Change any wording to suit your voice, but keep the numbers: each one comes from our own runs.

Opening and the problem (Harshavardhan, 0:00–1:30)

On screen: slide 1, then slide 2.

Harshavardhan: Hello everyone, I'm Harshavardhan Mallela.

Rohit: I'm Rohit Arya.

Shloak: And I'm Shloak Sinha.

Harshavardhan: This is our Computer Networks project, guided by Anuradha G. We built RUDP, a reliable transport protocol, from scratch in C on top of UDP, and then tested it under heavy packet loss.

Slide 2.

Harshavardhan: Start with the problem. When data crosses a network, packets get lost, duplicated and reordered, and nobody tells you. UDP behaves exactly like that. TCP fixes it: it numbers every segment, confirms what arrived, resends what didn't, and slows down when the network is struggling.

The slowing down matters more than it sounds. In October 1986 the internet had its first congestion collapse. On one path, between Lawrence Berkeley Laboratory and UC Berkeley, throughput fell from 32 kilobits per second to 40 bits per second. A sender that only retransmits can make congestion worse.

So our question was: can we build TCP's guarantees ourselves, on top of UDP, and prove they work?

What we built (Harshavardhan, 1:30–3:30)

On screen: slides 3, 4 and 5.

Harshavardhan: (slide 3) RUDP has five parts. A handshake to open a connection. Sequence numbers and checksums so every packet can be identified and checked. Retransmission, by timeout and by fast retransmit, to repair losses. A sliding window so several packets can be in flight at once. And AIMD congestion control to decide how fast to send. It is all written in C on plain UDP sockets.

(slide 4) Every packet starts with a 13-byte header: four bytes of sequence number, four of acknowledgment number, one byte of flags, two of checksum and two for the payload length. The flags are SYN, ACK, FIN and DATA, and they combine, so a SYN-ACK is simply two flags set together. We pack the fields byte by byte instead of sending a C struct, so compiler padding can never change what goes on the wire.

(slide 5) A connection opens with a three-way handshake: the client sends SYN, the server replies SYN-ACK, and the client confirms with ACK. Then data flows. Each data packet carries a sequence number, and the server answers with a cumulative ACK that means "I have everything before this number." When the client is done it sends FIN and the server closes the connection.

Reliability (Rohit, 3:30–4:30)

On screen: slide 6.

Rohit: Thanks, Harshavardhan. Now reliability. We used Go-Back-N. The sender keeps a window of packets in flight, and the receiver only accepts the packet it expects next. If something arrives out of order, the receiver discards it and repeats its last acknowledgment.

That repeated ACK is a signal. When the sender sees three duplicate ACKs, it knows a packet was probably lost and resends it immediately, without waiting. That is fast retransmit. As a safety net, if no new ACK arrives within 500 milliseconds, the sender times out and resends the whole window.

We checked this early: with 40 percent simulated loss, the server still reassembled our test message byte for byte.

Congestion control (Rohit, 4:30–5:45)

On screen: slide 7, pointing at the graph.

Rohit: Reliability alone is not enough, so we added congestion control, the same AIMD idea TCP uses. The window starts at one packet. In slow start it grows by one for every ACK, so it roughly doubles each round trip, until it reaches a threshold of sixteen. After that, in congestion avoidance, it grows by about one packet per round trip.

When loss happens, the response depends on how we noticed. A fast retransmit halves the window. A timeout, which is a worse sign, cuts the window back to one and starts again.

This graph shows it: the window climbs, loss hits, it drops, and it climbs again. That sawtooth is the signature of AIMD, and seeing it come out of our own code was a great moment for us.

Testing and results (Shloak, 5:45–7:45)

On screen: slide 8, then slide 9.

Shloak: (slide 8) We tested in three ways. First, a loss simulator built into the client, which drops outgoing data packets at a rate we choose. Second, tc netem, a Linux tool that makes the operating system itself drop packets; we used 20 percent loss on the loopback interface. Third, Wireshark, to check independently what really crossed the wire instead of trusting our own logs.

In the capture, handshake packets and acknowledgments are 13 bytes, header only, and data packets are 53 bytes: the header plus 40 bytes of payload. Under netem loss, retransmissions came in bursts about 500 milliseconds apart, which is exactly our timeout. The client logged 12 timeouts in that run, and the transfer still completed.

(slide 9) For the benchmark we ran the same 62-packet transfer at seven loss levels, from zero to 40 percent, using the built-in simulator. Every run delivered all 62 packets. What changed was time. With no loss the transfer took about a tenth of a second. At 10 percent it took about three seconds, and at 40 percent about twenty.

The right-hand chart shows why. As loss rises, AIMD keeps shrinking the final window, from about 19 packets at zero loss to around two. The protocol trades speed for stability instead of hammering a struggling network.

Live demo (Harshavardhan drives, Rohit and Shloak narrate, 7:45–9:00)

On screen: screen recording of two terminals side by side, server on the left and client on the right. Before recording, run `tc qdisc show dev lo` to confirm no netem rule is active, and restart the server.

```
# left terminal
cd ~/projects/rudp && make
./bin/server

# right terminal
./bin/client          # clean run
./bin/client 20       # 20% simulated loss
```

Harshavardhan: Let's see it run. On the left, the server is listening on port 8888. On the right, I'll start the client.

Run `./bin/client`.

Rohit: You can see the handshake finish, then the window climbing one packet at a time up to sixteen, and then the slower, linear growth.

Harshavardhan: It ends with all 62 packets delivered. Now let's make the network hostile: I'll tell the client to drop 20 percent of its packets.

Run `./bin/client 20`.

Shloak: Watch for the timeout lines. Each one resets the window to one and the sender resends. It is slower, but look at the server on the left: the full 2,442-byte message is reassembled, complete.

Problems we hit and how we fixed them (Shloak, 9:00–10:00)

On screen: slide 10.

Shloak: Not everything worked the first time, and the bugs taught us the most.

First, the server only accepted a new connection while in the LISTEN state. After a client was killed mid-transfer, the server stayed stuck and ignored every later handshake. Our benchmark failed identically at every loss level, each run taking about 15 and a half seconds: five handshake retries at three seconds each. The fix was to accept a fresh SYN in any state.

Second, a leftover netem rule from an earlier capture silently stacked on top of the next test and made a healthy run look hung. Now we always check with `tc qdisc show` and delete the rule afterwards.

Third, Wireshark on Windows could not see traffic inside WSL's loopback, so we captured with tshark inside WSL and opened the file in Wireshark.

Fourth, the timing and regex commands in our benchmark script misbehaved on this shell, so we replaced them with portable awk and sed.

Literature and application area (Rohit, 10:00–11:00)

On screen: slide 11.

Rohit: Our design follows a well-established line of research. Jacobson and Karels' 1988 paper, "Congestion Avoidance and Control," introduced slow start and congestion avoidance after the 1986 collapse. Chiu and Jain showed why additive increase with multiplicative decrease converges to a fair and efficient share. RFC 5681 standardised these algorithms. CUBIC, by Ha, Rhee and Xu, replaces linear growth with a cubic function, and it is what we want to compare against next. And QUIC, from Google, shows the same idea we used, reliable transport built on UDP, running at internet scale.

The researcher we want to highlight is Van Jacobson. Every congestion-control mechanism in RUDP traces back to his work.

Our application area is user-space reliable transport over UDP, the same architecture QUIC uses, and our protocol doubles as a testbed for studying how transports behave under loss.

Lab topology (Shloak, 11:00–11:45)

On screen: slide 12, then a cut to the Cisco Packet Tracer window showing a successful ping. Record this part after the topology is built.

Shloak: For the Cisco part of the lab, we designed a topology with two subnets joined by a router. The client sits on 192.168.1.0/24 behind one switch, the server on 192.168.2.0/24 behind another, and the router's two interfaces act as the gateways.

Cut to Packet Tracer: ping from PC-Client to PC-Server.

Shloak: The ping crosses the router, which is the network path RUDP traffic would take if we moved from loopback to separate machines.

Learnings, future work and close (all three, 11:45–13:15)

On screen: slide 13, then slide 14.

Harshavardhan: So what did we learn? Reliability is mostly timers and edge cases, not the happy path.

Rohit: Congestion control is a feedback loop. It is not a speed limit; it is the sender listening to the network.

Shloak: And independent evidence matters. Wireshark confirmed what our logs claimed, and debugging the environment mattered as much as debugging the code.

Harshavardhan: Our protocol has limits. It uses Go-Back-N, so one lost packet resends the whole window. Its timeout is a fixed 500 milliseconds instead of being estimated from measured round-trip times. And we tested on one machine with emulated loss, not a real wide-area network.

Rohit: That points to the next steps: Selective Repeat or SACK so only lost packets are resent, an adaptive timeout based on measured round-trip time, a CUBIC variant to compare with our AIMD results, and tests across the Cisco topology.

Shloak: We would also like to explore learning-based congestion control.

Slide 14.

Harshavardhan: Thank you for watching. The code, report and benchmarks are on GitHub at Capsturrr slash rudp.

All three: Thank you!

Recording checklist and a shorter cut

- ☐ Check that the faculty name (Anuradha G) is right on slide 1 and in the opening line.
- ☐ Run `tc qdisc show dev lo`; delete any netem rule with `sudo tc qdisc del dev lo root netem`.
- ☐ Restart `./bin/server` fresh and run `make` once to show a clean build.
- ☐ Build the Packet Tracer topology and screenshot the working ping; swap it into slide 12 or cut to it.
- ☐ Use a large terminal font; close tabs and windows that show tokens or personal data.

- ☐ Record each speaker's part as its own clip, then join them in the editor.
- ☐ Upload to YouTube, put the GitHub link and slide titles in the description, and share the link with your faculty.

Shorter cut (about 8 minutes): merge slides 4 and 5 into one sentence, keep two bugs on slide 10, trim the literature to Jacobson and QUIC, and cut the topology segment to 20 seconds.